PROGRESS PRESENTATION · MONDAY 19 OCTOBER 2026

LangNation Cloud Platform

Milestones 1–6 · implemented and validated

AWS Cloud Computing with AI Bootcamp

What I built, why I designed it this way, and how I validated it.



Request path · secure · highly available · automated · containerized · cloud-native

Presented by Seraj Ahmad Siddiqui

Guidance: Micheal (Theory Teacher) · Rahul (Practical Teacher) · Dominic (HoD)

6 milestones
implemented and validated

2 AZs
multi-AZ application + data layer

Auto recovery
morning rebuild after nightly cleanup + DNS sync

Live endpoint
cloud.namienterprises.in

The challenge and the design goal

Overview

Real LangNation use case: students learn, teachers manage materials, admins operate the platform — on a repeatable, secure AWS foundation.

CHALLENGE 1

Sandbox resets

AWS resources are cleaned regularly, so the environment must be reproducible instead of manually rebuilt.

CHALLENGE 2

Security

No direct access to ECS or RDS; edge traffic passes CloudFront + WAF; authentication uses Cognito JWTs.

CHALLENGE 3

Availability

Application tasks, networking and database components span two Availability Zones.

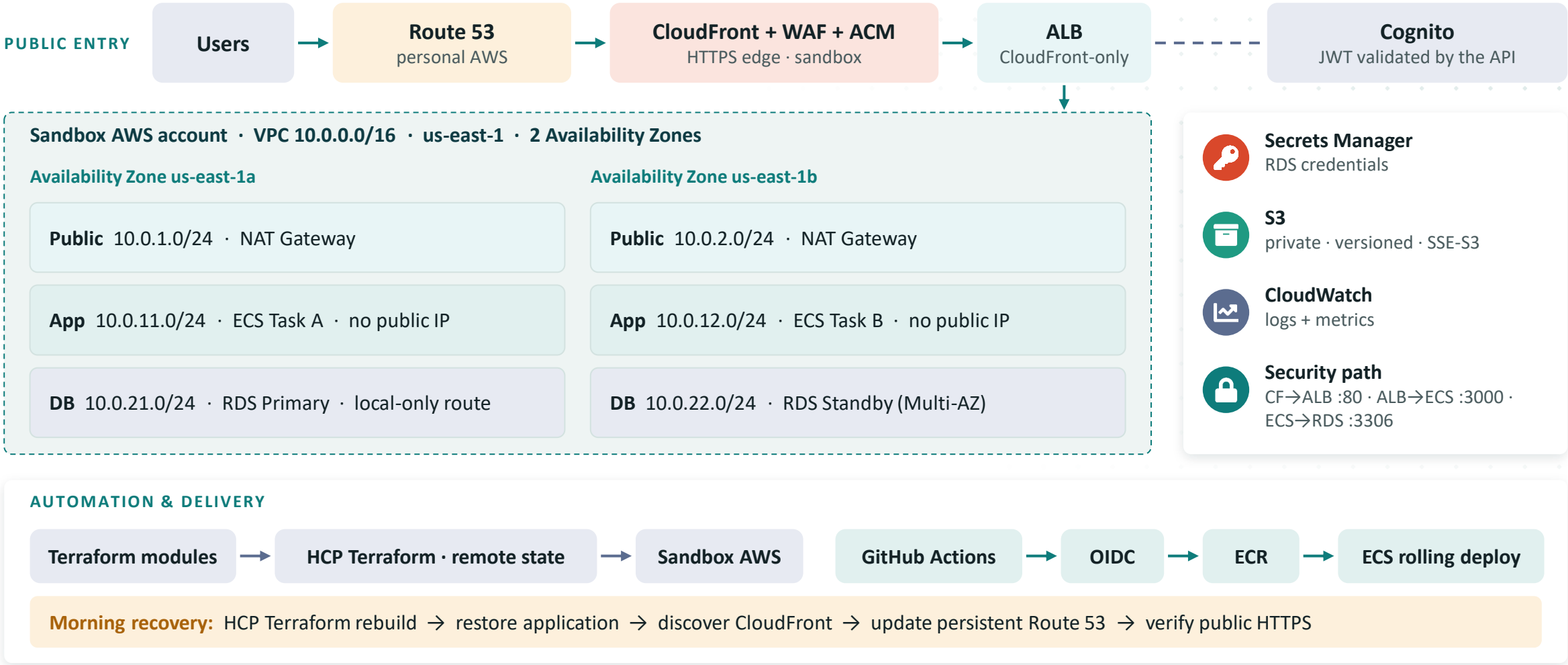
Target platform

Reproducible	Terraform / HCP Terraform owns infrastructure
Deployable	GitHub Actions builds and releases the application
Private by default	ECS and RDS stay in private subnets
Protected at the edge	HTTPS + CloudFront + WAF
Authenticated	Cognito access tokens protect API routes
Recoverable	Morning automation rebuilds infra, app and DNS

Master architecture — Milestones 1–6

M1–M6

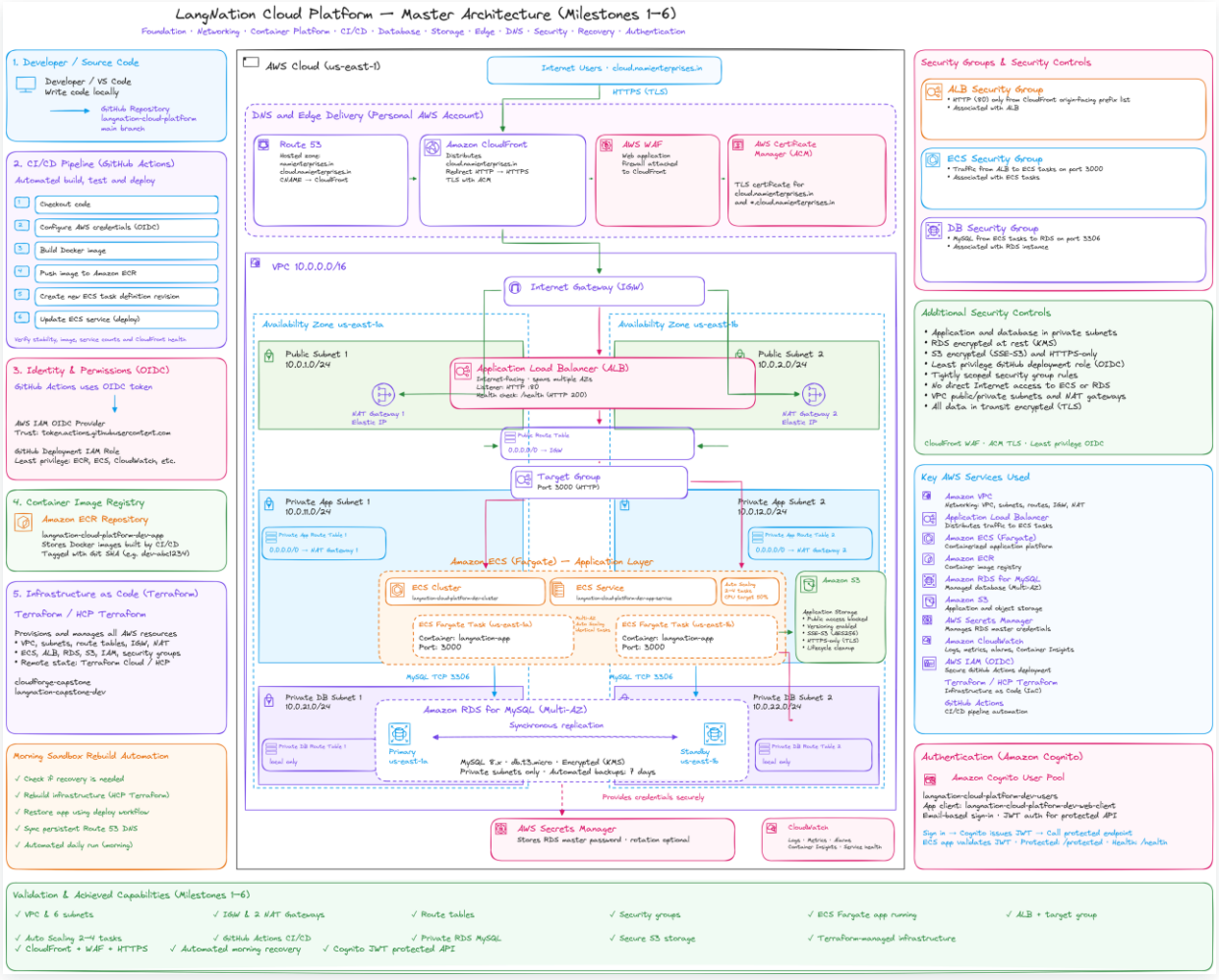
Everything completed so far, on one map. Use it as the guide for the rest of the presentation.



Master architecture — full detail

M1–M6

The complete Milestones 1–6 reference diagram, from source code to the private data tier.



How to read it

Left · delivery & operations

GitHub Actions + OIDC, ECR, Terraform / HCP and morning recovery

Top · DNS & edge

Route 53 → CloudFront + WAF + ACM → ALB origin

Centre · VPC 10.0.0.0/16

Public, private app and private DB tiers across two AZs

Right · security & services

Security groups, Cognito, Secrets Manager, S3, CloudWatch

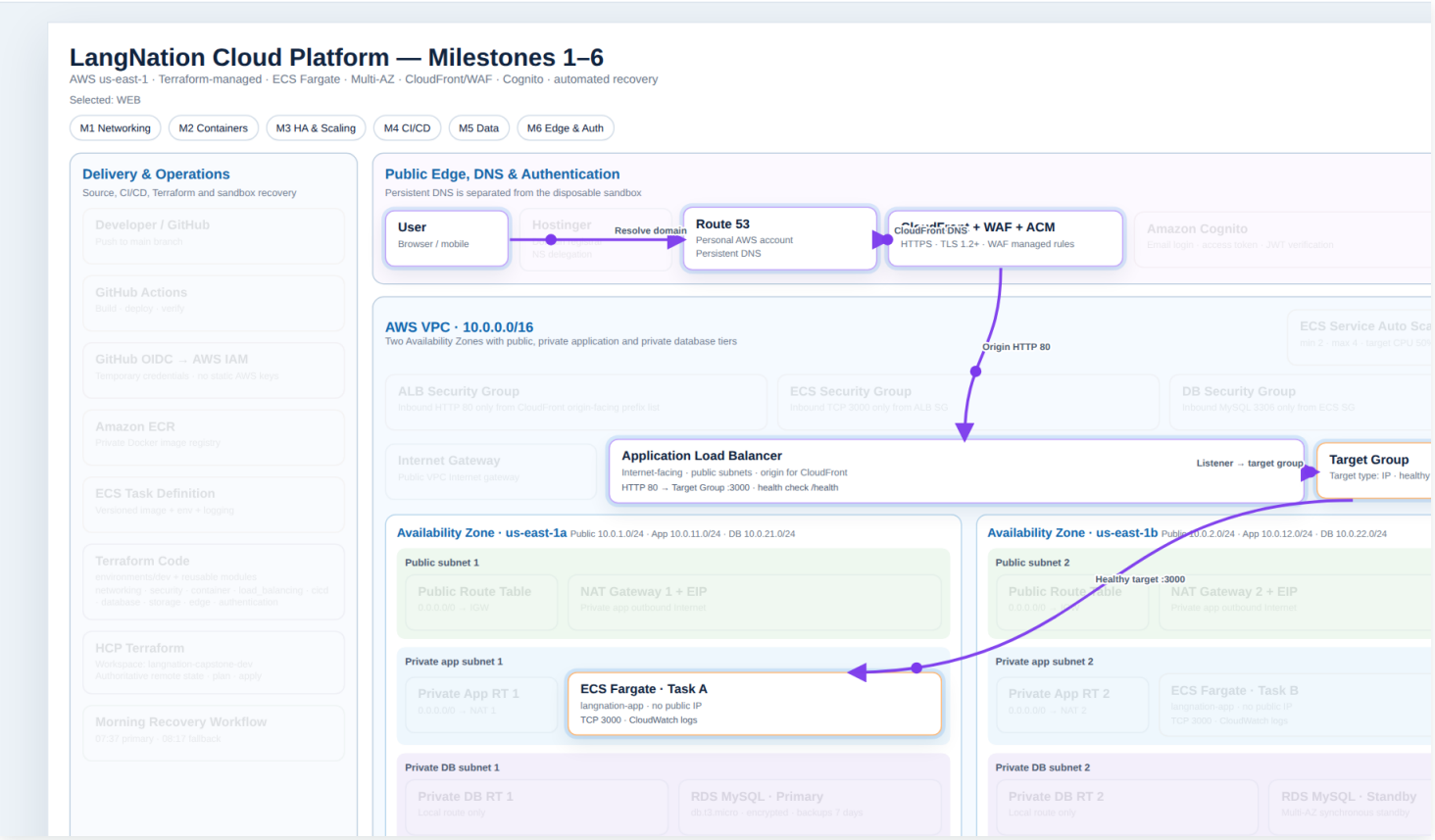
Bottom · validated

Checklist of capabilities delivered in M1–M6

Live interactive walkthrough

Live demo

An animated version of the architecture — choose a milestone or flow and watch the request path move.



Shown: Flow · Public web request — User → Route 53 → CloudFront + WAF → ALB → target group → ECS task.

Live walkthrough

Opens the HTML file in your browser. Keep it in the same folder as this deck.

Suggested demo order

1. Milestones M1 → M6 (keys 1–6)
2. Flow · CI/CD deployment
3. Flow · Cognito authentication
4. Flow · Morning sandbox recovery

Keyboard

`1–6`	milestone	`Space`	play / pause
`A`	all flows	`F`	fullscreen
`S`	auto sequence		

Foundation & networking

M1

A two-AZ VPC separates public ingress, application compute and the database layer.

VPC 10.0.0.0/16

	Availability Zone us-east-1a	Availability Zone us-east-1b
Public internet-facing	**10.0.1.0/24** ALB · NAT Gateway	**10.0.2.0/24** ALB · NAT Gateway
Private app compute	**10.0.11.0/24** ECS Fargate tasks	**10.0.12.0/24** ECS Fargate tasks
Private DB data	**10.0.21.0/24** RDS only · no internet route	**10.0.22.0/24** RDS only · no internet route

Routing design

Public	→ Internet Gateway
Private app	→ same-AZ NAT
Private DB	→ local only

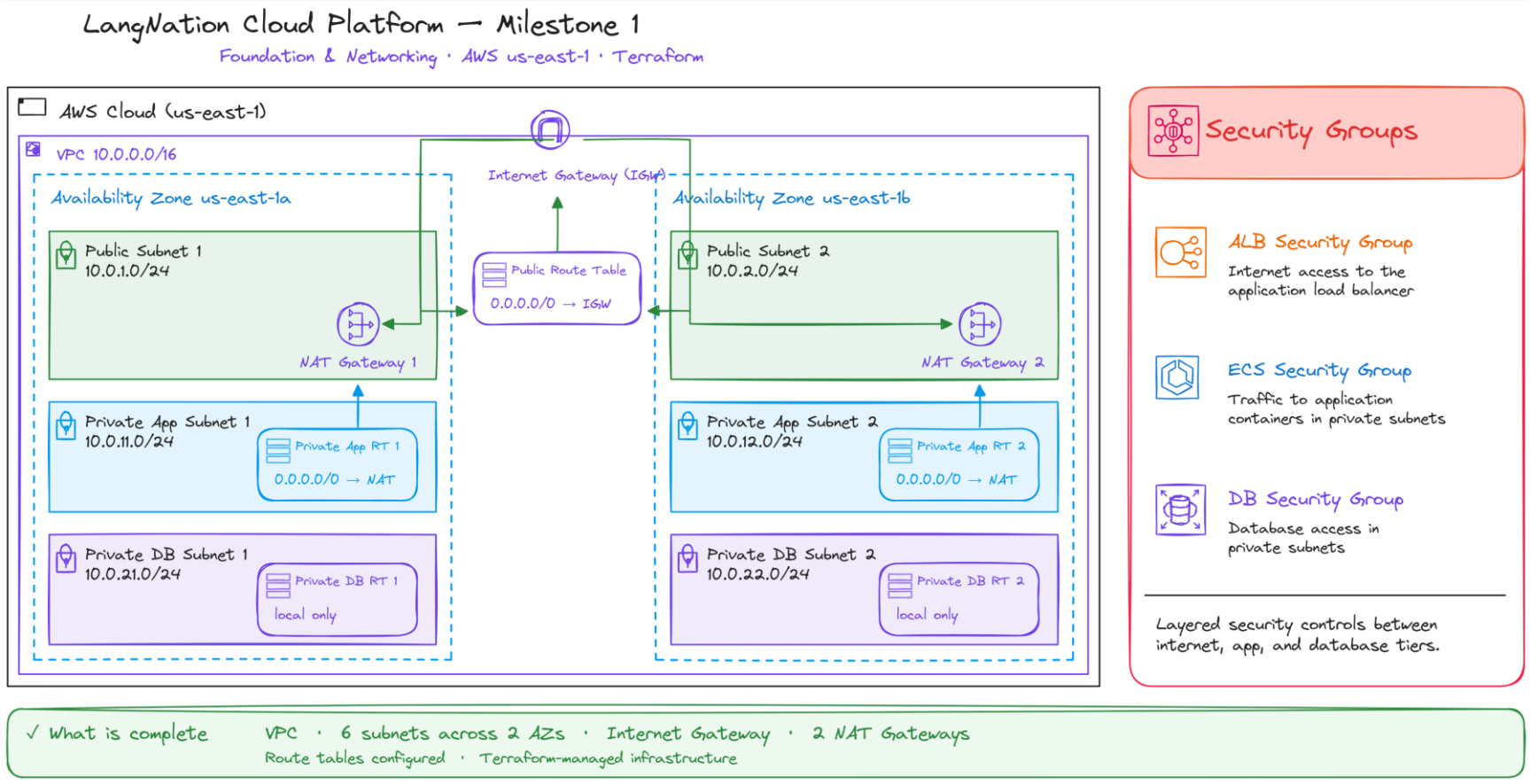
No public IP on ECS
tasks are reachable only through the ALB

RDS not public
database has no internet route

Foundation & networking — as built

M1

Milestone 1 delivered the VPC, six subnets, per-tier route tables and layered security groups — all from Terraform.



Milestone 1 diagram — each tier has its own route table; each AZ has its own NAT gateway.

6

subnets across two Availability Zones

2

NAT gateways, one per AZ

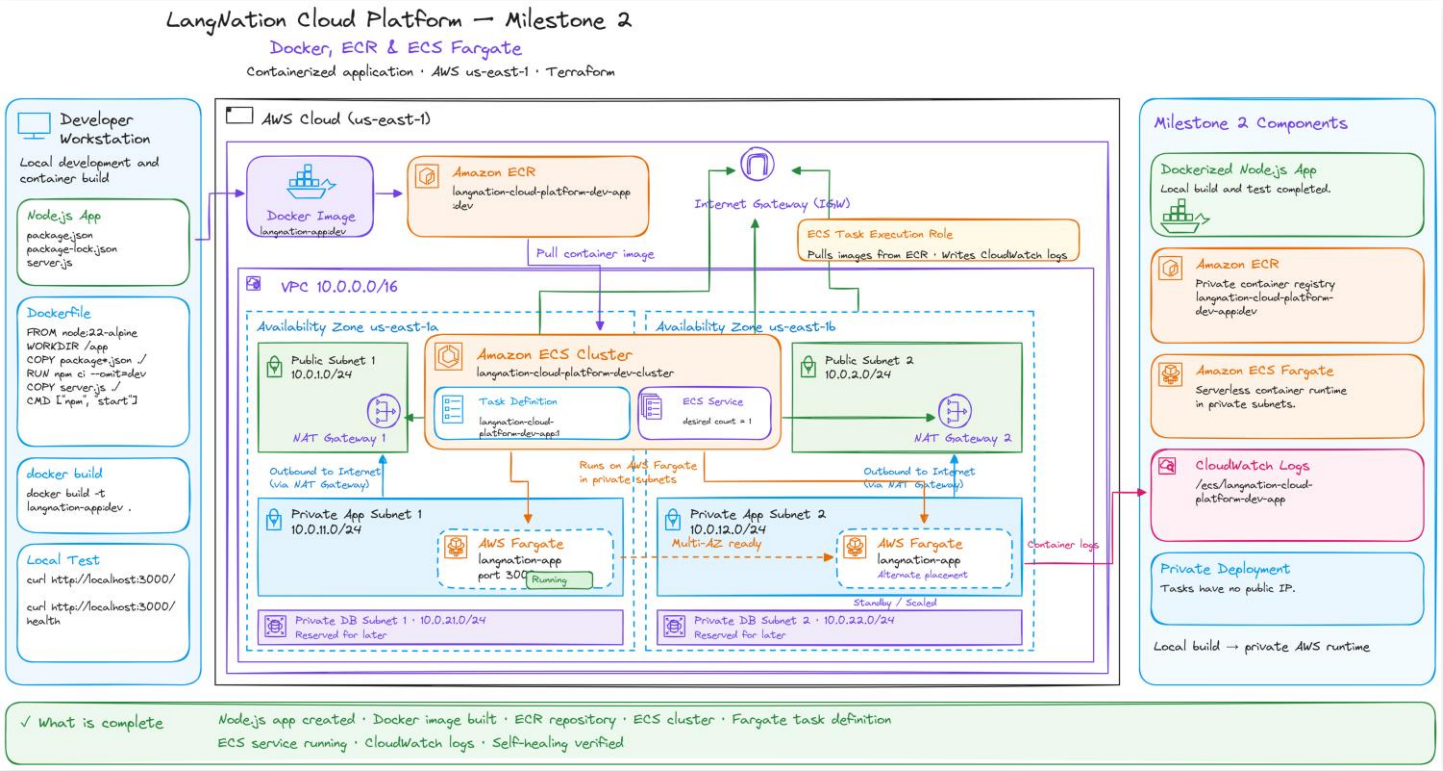
3

security groups: ALB · ECS · DB

Containers, load balancing & auto scaling

M2 · M3

Docker images are stored in ECR and deployed as private ECS Fargate tasks behind an Application Load Balancer.



Milestone 2 build snapshot (one task at that stage). The service now runs two tasks with auto scaling, as summarised on the right.

Baseline capacity

2 ECS tasks spread across both Availability Zones

Auto scaling

Min 2 · max 4 · CPU target 50%

Health & self-healing

ALB checks /health on port 3000; failed tasks are replaced

Controlled validation

CPU threshold test scaled the service from 2 → 4 tasks

CI/CD and Infrastructure as Code

M4

GitHub Actions owns application releases; Terraform code + HCP Terraform own infrastructure and remote state.

Application deployment — GitHub Actions

No long-lived AWS keys · GitHub OIDC

1. Push to main
2. OIDC to AWS
3. Build Docker
4. Push to ECR
5. Render task definition
6. Deploy ECS
7. Verify SHA + counts
8. Health via CloudFront

Infrastructure as Code — Terraform + HCP Terraform

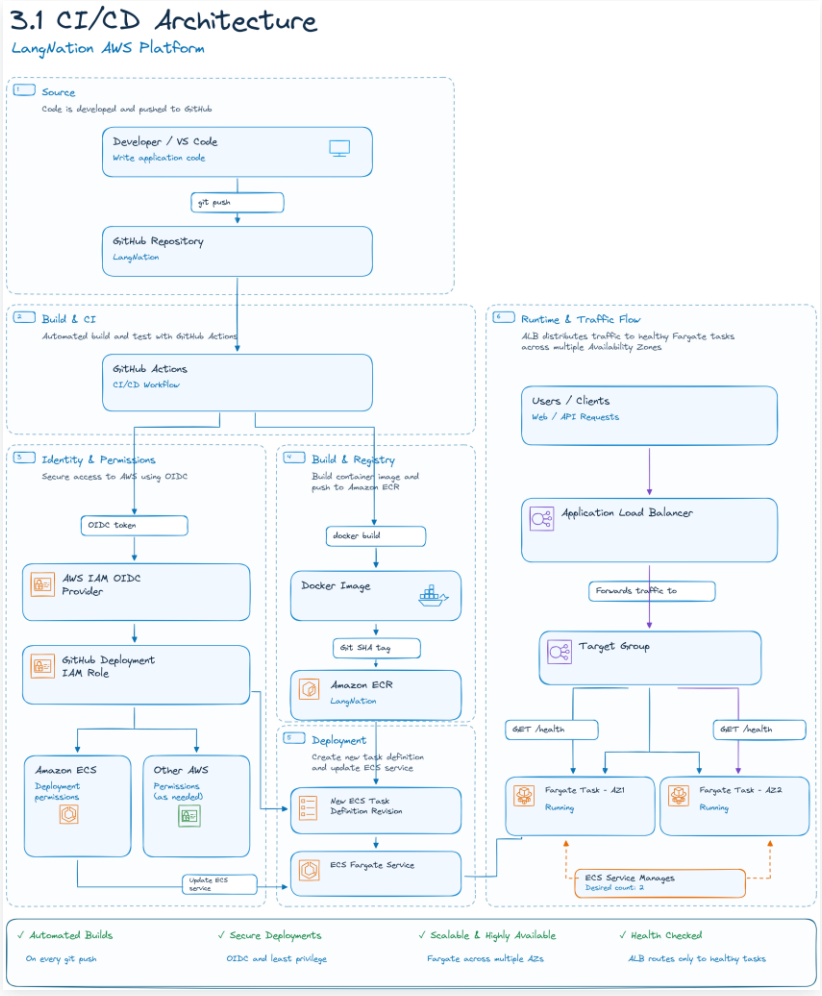
1. Commit Terraform changes
2. HCP Terraform plan
3. Review / apply
4. Authoritative remote state
5. Reproducible sandbox rebuild

Clear ownership: Terraform owns the infrastructure · GitHub Actions owns application task-definition revisions

CI/CD pipeline — end to end

M4

From a git push to healthy Fargate tasks in both Availability Zones — six stages, no static AWS keys.



1 Source

Developer pushes to the main branch of the GitHub repository

2 Build & CI

GitHub Actions workflow starts automatically on every push

3 Identity

OIDC token → IAM OIDC provider → deployment role; no static keys

4 Build & registry

Docker image built, tagged with the Git SHA and pushed to ECR

5 Deployment

New task-definition revision; the ECS Fargate service rolls it out

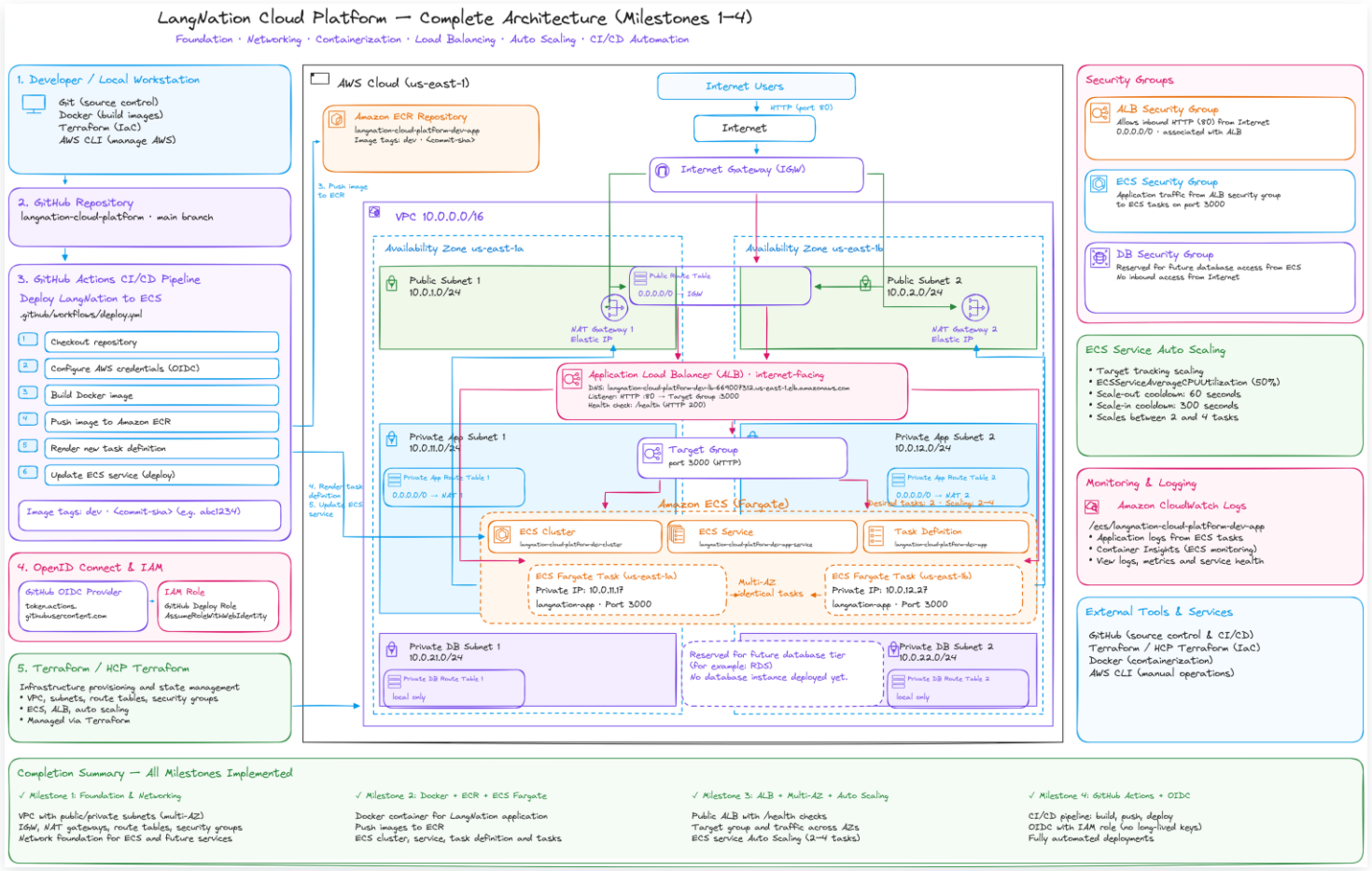
6 Runtime

ALB → target group → healthy Fargate tasks in both AZs (desired 2)

Checkpoint — Milestones 1–4 complete

M1–M4

Where the platform stood before data and edge security were added.



Snapshot before M5–M6: the ALB was still internet-facing and the DB subnets were empty.

M1 — Foundation & networking

VPC, 6 subnets in 2 AZs, IGW, NAT gateways, route tables

M2 — Docker, ECR & Fargate

Containerised app, private ECR, ECS cluster and service

M3 — ALB, Multi-AZ & scaling

/health checks, tasks in both AZs, auto scaling 2–4

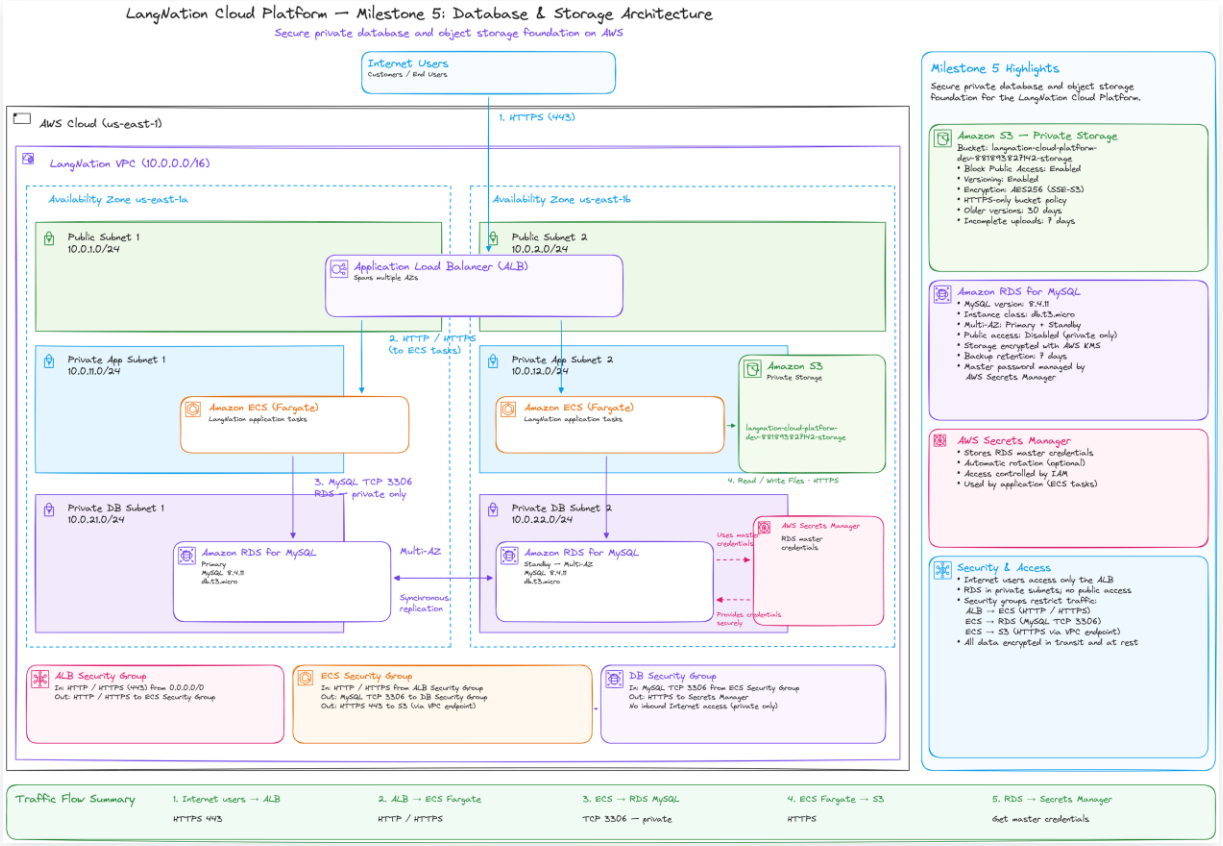
M4 — GitHub Actions + OIDC

Build, push and deploy with no long-lived AWS keys

Database & storage

M5

The data layer is private, encrypted and designed for availability.



Milestone 5 snapshot — ALB ingress was later locked to CloudFront only in M6.

RDS MySQL Multi-AZ

db.t3.micro · encrypted · private · 7-day backups · managed master secret

Network isolation

DB security group accepts TCP 3306 only from the ECS security group

Amazon S3

Private bucket · block public access · versioning · SSE-S3 · HTTPS-only policy

Lifecycle controls

Non-current versions expire after 30 days; incomplete multipart uploads abort after 7 days

Edge security, DNS & authentication

M6

Milestone 6 turned the platform into a protected public service with authenticated API access.



HTTPS edge

ACM certificate; CloudFront redirects HTTP → HTTPS and reaches the ALB origin over HTTP 80.

AWS WAF

AWS managed common rule set plus per-IP rate limiting.

ALB hardening

Port 80 accepts only the CloudFront origin-facing managed prefix list — no bypass.

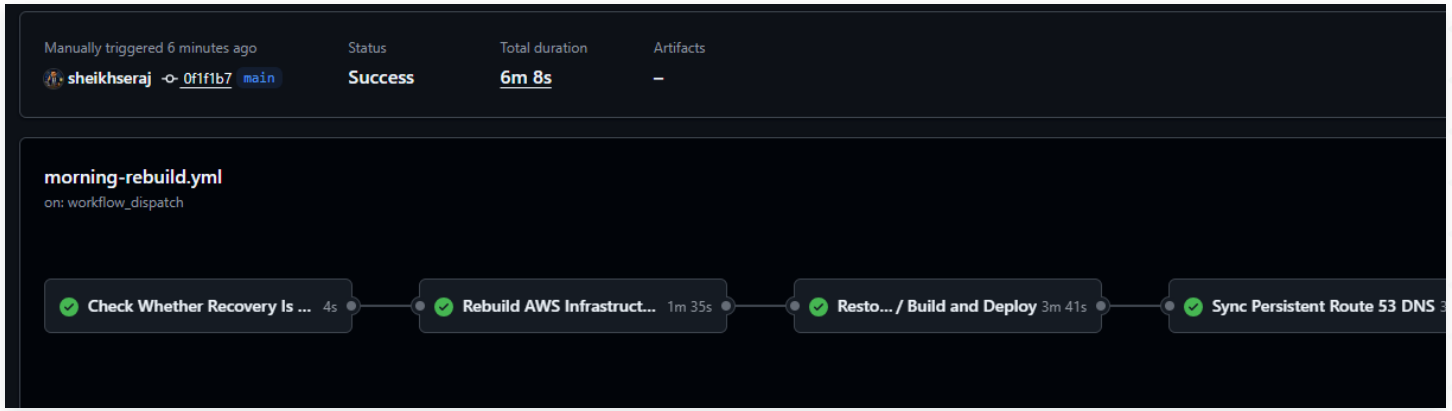
Cognito + JWT

/health stays public; /protected returns 401 without a token and 200 with a valid access token.

Automated morning recovery

Operations

The sandbox is cleaned overnight, so recovery is an operational workflow — validated end-to-end.



Run #6 — manual end-to-end validation: all four jobs green, total duration 6 m 8 s.

Primary schedule

07:37

Europe/Berlin time

Fallback schedule

08:17

Skips if the endpoint is already healthy

Why personal Route 53?

DNS must persist even when sandbox CloudFront / ACM resources are recreated.

Recovery sequence

1. Check whether recovery is needed
2. HCP Terraform rebuilds infrastructure
3. Deploy current application image
4. Read new CloudFront domain
5. OIDC into personal AWS account
6. UPSERT Route 53 CNAME
7. Verify public HTTPS /health

Evidence: tested, not just configured

Validation

These are the outcomes I would demonstrate if there is time for a live demo.

200

Normal HTTPS request

```
GET /health
→ {"status":"healthy"}
```

403

WAF blocks bad input

```
XSS-style query
→ request blocked
```

TIMEOUT

Direct ALB bypass

```
Public ALB DNS
→ connection blocked
```

401 → 200

Protected API

```
No JWT → 401
Valid Cognito JWT → 200
```

Additional validation completed

- ECS service: desired 2 / running 2 / pending 0
- Targets healthy in both private app subnets
- Auto scaling test: 2 → 4 tasks
- CloudFront status: Deployed
- ACM certificate: ISSUED / validation SUCCESS
- Morning recovery: manual end-to-end run, all jobs green

Engineering challenges I solved

Lessons

The capstone included real troubleshooting — not only successful “happy path” deployments.

01 Nightly sandbox cleanup

FIX Built Terraform / HCP reproducibility plus a morning recovery pipeline.

02 Route 53 must survive cleanup

FIX Moved authoritative DNS to personal AWS and automated cross-account CNAME sync.

03 GitHub OIDC trust initially failed

FIX Restricted trust to immutable repository / owner IDs and the main branch.

04 ECS action reported a failed deployment

FIX Validated by service stability + exact Git SHA + task counts + CloudFront health.

05 ALB could bypass WAF

FIX Reworked the CI/CD health check, then restricted ALB ingress to CloudFront’s prefix list.

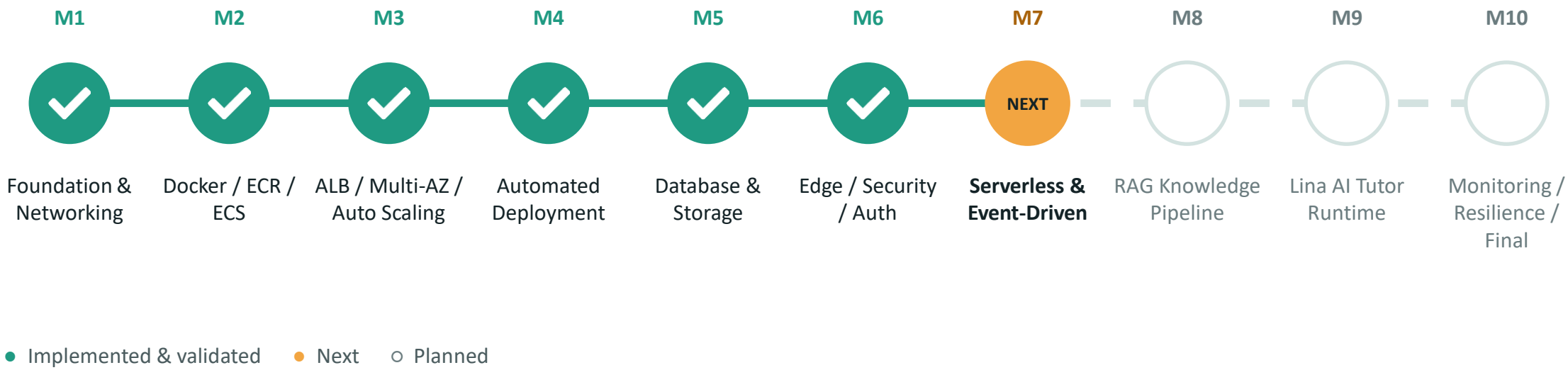
06 PowerShell / AWS CLI parsing issues

FIX Used safer quoted CLI arguments and validated every step independently.

Where the project stands now

Roadmap

Milestones 1–6 are implemented and validated. Next: Serverless & Event-Driven, followed by the AI/RAG milestones.



Next: Milestone 7 — Serverless & Event-Driven Architecture

Then RAG → AI Tutor → monitoring, resilience, cost and final documentation

20 Oct 2026

target

Key takeaway

The platform is now secure, multi-AZ, reproducible and operationally recoverable.

Secure by design

Private compute and data, WAF, CloudFront-only ALB, Cognito JWT

Highly available

Two AZs, multiple ECS tasks, auto scaling, Multi-AZ RDS

Automated

Terraform + HCP Terraform + GitHub Actions + scheduled recovery

Demonstrably working

200 / 403 / timeout / 401 / authenticated 200 tests

“Built from code. Protected at the edge. Recovered automatically.”

cloud.namienterprises.in

Thank you · AWS Cloud Computing with AI Bootcamp

Acknowledgements & support

Thanks

AWS Cloud Computing with AI Bootcamp · guidance, feedback and peer support behind this capstone.

Instruction & leadership

Micheal — Theory Teacher
Rahul — Practical Teacher
Dominic — Head of Department

Supporting teachers

Shahabuddin · Charisima
and the wider teaching team

Peer & student support

All of my classmates — discussions, feedback, testing and motivation throughout the capstone

Questions?

Thank you for the guidance, questions and support that helped me improve the project.

cloud.namienterprises.in

Backup — key technical configuration

BACKUP

Keep this slide for Q&A — do not present unless asked.

VPC	10.0.0.0/16	**RDS**	MySQL 8.4.11 · db.t3.micro · Multi-AZ · 20 GiB gp3
Public subnets	10.0.1.0/24 · 10.0.2.0/24	**Backups**	7 days
Private app	10.0.11.0/24 · 10.0.12.0/24	**S3**	versioned · SSE-S3 · HTTPS-only
Private DB	10.0.21.0/24 · 10.0.22.0/24	**WAF**	AWS Managed Common Rule Set + 1000/IP rate rule
ECS	Fargate · 256 CPU · 512 MiB · port 3000	**Auth**	Cognito User Pool + public app client (no secret)
Scaling	min 2 · max 4 · CPU target 50%	**Domain**	cloud.namienterprises.in · Route 53 in personal AWS

Backup — likely questions & short answers

BACKUP

Use this as your mental checklist during Q&A.

Why personal Route 53?

The sandbox is disposable. Route 53 stays in personal AWS so DNS, mail records and ACM validation remain stable across nightly cleanup.

Why two NAT gateways?

Each private app subnet has same-AZ egress, reducing an AZ dependency.

Why OIDC instead of AWS keys?

Short-lived federated credentials; no long-lived GitHub AWS secrets.

Why restrict the ALB?

Otherwise users could bypass CloudFront and WAF using the ALB DNS name.

Why HCP Terraform?

Remote state, controlled plan/apply, and reproducible rebuilds.

Why Cognito?

Managed identity service; the API validates signed access JWTs instead of implementing identity storage itself.